

Scalable Data Processing

Distributed Data Processing

Niteesh K R

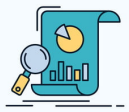
Computer Applications

Scalable Data Processing

Distributed Data Processing

Data Engineering – Introduction, Roles, Lifecycle

Niteesh K R
Computer Applications

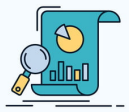


Scalable Data Processing

Introduction to Data Engineering

PES UNIVERSITY

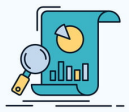
- Data Engineering is the discipline focused on designing, building, and maintaining systems that collect, store, process, and deliver data
- It acts as the foundation layer for analytics, machine learning, and decision-making systems
- Unlike traditional application development, Data Engineering focuses on data flow, scalability, reliability, and performance
  - Concerned with how data moves from source systems to analytical platforms
  - Ensures data is available, clean, consistent, and timely
  - Works closely with storage systems, distributed processing, and streaming platforms



Scalable Data Processing

Why Data Engineering is Needed

- Modern organisations generate massive volumes of data from multiple sources
- Raw data is often noisy, incomplete, and unsuitable for direct analysis
- Data Engineering bridges the gap between raw data and usable insights
  - Handles data from databases, logs, sensors, APIs, and user interactions
  - Prepares data for downstream systems such as BI tools and ML models
  - Enables scalability when data grows from MBs to TBs or PBs

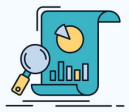


Scalable Data Processing

Evolution of Data Engineering

PES UNIVERSITY

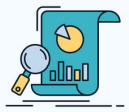
- Data Engineering has evolved alongside data storage and processing technologies
- Early systems relied on single-machine relational databases
- Growth of internet-scale data led to distributed systems
  - Traditional ETL pipelines using RDBMS and data warehouses
  - Big Data era with Hadoop, HDFS, and MapReduce
  - Modern pipelines using Spark, Flink, Kafka, and cloud-native systems



Scalable Data Processing

Data Engineering vs Data Science

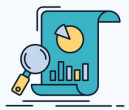
- Data Engineering and Data Science are complementary but distinct roles
- Data Engineers focus on infrastructure and pipelines
- Data Scientists focus on analysis, modelling, and interpretation
  - Data Engineers build reliable data pipelines
  - Data Scientists consume curated datasets
  - Poor Data Engineering directly impacts model accuracy and insights



Scalable Data Processing

Types of Data Handled in Data Engineering

- Data Engineers work with multiple data formats and structures
- Each type of data requires different storage and processing strategies
  - Structured data (tables, CSV, relational databases)
  - Semi-structured data (JSON, XML, logs)
  - Unstructured data (text, images, videos)

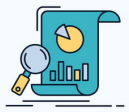


Scalable Data Processing

Data Engineering Ecosystem

PES UNIVERSITY

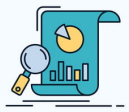
- Data Engineering uses a combination of storage, processing, and orchestration tools
- Tools are selected based on volume, velocity, and variety of data
  - Storage systems such as HDFS and distributed file systems
  - Processing engines such as Spark and Flink
  - Workflow orchestration using tools like Airflow



Scalable Data Processing

What is a Data Pipeline

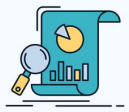
- A data pipeline is a sequence of steps that moves data from source to destination
- Pipelines automate ingestion, transformation, and storage
- They must be reliable, scalable, and fault-tolerant
  - Ingest raw data from multiple sources
  - Transform data into usable formats
  - Load processed data into analytical systems



Scalable Data Processing

Batch vs Streaming Pipelines

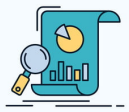
- Data pipelines can be classified based on how data is processed
- Batch processing works on large chunks of historical data
- Streaming processing works on continuous data flows
  - Batch pipelines are suitable for periodic reporting
  - Streaming pipelines enable real-time analytics
  - Modern systems often combine both approaches



Scalable Data Processing

Who is a Data Engineer

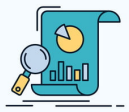
- A Data Engineer is responsible for building and maintaining data infrastructure
- They ensure data availability, quality, and performance
- The role requires strong understanding of systems and data flow
  - Works closely with Data Scientists and Analysts
  - Designs scalable data architectures
  - Monitors and optimises pipeline performance



Scalable Data Processing

Core Responsibilities of a Data Engineer

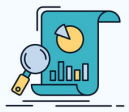
- Data Engineers handle the end-to-end data movement and preparation
- Responsibilities span development, deployment, and maintenance
  - Data ingestion from multiple sources
  - Data cleaning and transformation
  - Managing distributed storage and compute systems



Scalable Data Processing

Skills Required for a Data Engineer

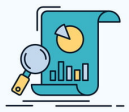
- Data Engineers require a mix of programming, systems, and data skills
- Focus is more on reliability than experimentation
  - Strong programming skills (Python, SQL)
  - Understanding of distributed systems
  - Familiarity with Big Data frameworks



Scalable Data Processing

Data Engineering Lifecycle

- The Data Engineering Lifecycle defines how data flows through systems
- It ensures structured handling of data from source to consumption
- Each phase has clear objectives and challenges
  - Data ingestion
  - Data storage
  - Data processing and transformation
  - Data serving and consumption

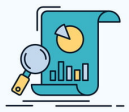


Scalable Data Processing

Lifecycle Phase 1: Data Ingestion

PES UNIVERSITY

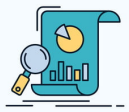
- Data ingestion is the process of collecting data from source systems
- Sources may be structured or unstructured
- Ingestion must handle scale and failures
  - Databases, APIs, logs, sensors
  - Batch ingestion using scheduled jobs
  - Streaming ingestion using message queues



Scalable Data Processing

Lifecycle Phase 2: Data Storage

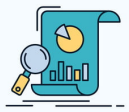
- Stored data must be durable, scalable, and cost-effective
- Storage choices depend on access patterns
  - Distributed file systems such as HDFS
  - Columnar formats for analytics
  - Separation of raw, processed, and curated data



Scalable Data Processing

Lifecycle Phase 3: Data Processing

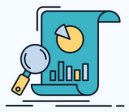
- Processing converts raw data into meaningful information
- Can be batch-based or stream-based
- Focus on performance and correctness
  - Data cleansing and validation
  - Aggregations and joins
  - Business logic application



Scalable Data Processing

Lifecycle Phase 4: Data Serving

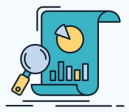
- Processed data must be made available to consumers
- Serving layer supports analytics and applications
  - Data warehouses and analytical databases
  - Dashboards and reporting tools
  - Machine learning pipelines



Scalable Data Processing

Data Quality and Monitoring

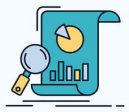
- Data quality is critical for trust and decision-making
- Pipelines must be monitored continuously
  - Schema validation and consistency checks
  - Detecting missing or late data
  - Monitoring pipeline failures and delays



Scalable Data Processing

Example: Simple Data Pipeline

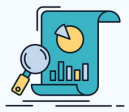
- Consider a pipeline that processes application logs
- Logs are ingested, stored, processed, and analysed
  - Logs ingested daily from servers
  - Stored in distributed storage
  - Aggregated to identify usage patterns



Scalable Data Processing

Experiential Learning – Banana

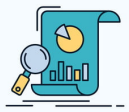
- Scenario: A college collects attendance data daily in CSV format
- Problem: Identify the stages of the Data Engineering lifecycle for this system
- Solution:
  - Ingestion: Daily CSV upload
  - Storage: Distributed file storage
  - Processing: Aggregation of attendance
  - Serving: Reports for faculty



Scalable Data Processing

Experiential Learning – Orange

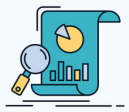
- Scenario: An e-commerce platform tracks user clicks in real time
- Problem: Design a high-level data pipeline and classify processing type
- Solution:
  - Streaming ingestion of click events
  - Real-time processing for metrics
  - Storage for historical analysis



Scalable Data Processing

Experiential Learning – Jackfruit

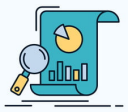
- Scenario: A fintech company processes millions of transactions daily
- Problem: Propose a complete Data Engineering lifecycle with fault tolerance
- Solution:
  - Streaming ingestion with retries
  - Distributed storage with replication
  - Batch and streaming analytics
  - Monitoring and alerting mechanisms



Scalable Data Processing
Brainstorming Questions

PES UNIVERSITY

- Why is Data Engineering critical before applying Machine Learning?
- What risks arise if data ingestion is unreliable?
- Can a system rely only on batch processing in modern applications?
- How does poor data quality impact business decisions?



Scalable Data Processing

References

- Fabian Hueske and Vasiliki Kalavri, *Stream Processing with Apache Flink*, O'Reilly Media, 2019 – Chapter 1: Introduction to Stream Processing
- Shilpi Saxena and Saurabh Gupta, *Practical Real-time Data Processing and Analytics*, Packt Publishing – Chapter 1: Big Data Processing Overview
- Apache Flink Documentation v1.20.0 – Concepts and Architecture
- PyFlink API Reference v1.20.0



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456



Scalable Data Processing

Distributed Data Processing

Niteesh K R

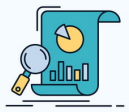
Computer Applications

Scalable Data Processing

Distributed Data Processing

Batch and Stream Processing, Distributed FS

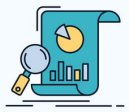
Niteesh K R
Computer Applications



Scalable Data Processing

Introduction to Data Processing Paradigms

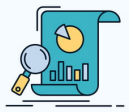
- Data processing refers to transforming raw data into meaningful information
- Based on how and when data is processed, systems follow different paradigms
- Two dominant paradigms in Data Engineering are Batch Processing and Stream Processing
  - Determines latency, architecture, and tool choice
  - Impacts scalability, fault tolerance, and cost
  - Forms the foundation for analytics and real-time systems



Scalable Data Processing

What is Batch Processing

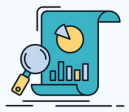
- Batch processing handles data in large volumes collected over a period of time
- Data is processed as a group rather than individually
- Processing typically happens at scheduled intervals
  - Suitable for historical and archival data
  - High throughput but higher latency
  - Widely used in reporting and offline analytics



Scalable Data Processing

Characteristics of Batch Processing

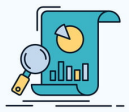
- Batch systems prioritise throughput over response time
- Results are available only after the batch completes
  - Data is finite and bounded
  - Processing is repeatable and deterministic
  - Failures can be retried using the same dataset



Scalable Data Processing

Use Cases of Batch Processing

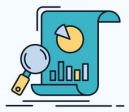
- Batch processing is suitable when real-time results are not mandatory
  - Payroll processing at the end of the month
  - Generating daily or weekly business reports
  - Historical trend analysis and data warehousing
- Batch systems are cost-efficient for large-scale offline computation



Scalable Data Processing

Limitations of Batch Processing

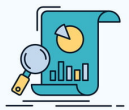
- Batch processing introduces inherent delay between data generation and insight
- Not suitable for time-sensitive applications
  - Cannot react immediately to events
  - Unsuitable for fraud detection or monitoring systems
  - Latency increases with batch size



Scalable Data Processing

What is Stream Processing

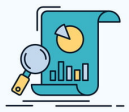
- Stream processing handles data continuously as it arrives
- Each record is processed independently or in small windows
- Enables near real-time or real-time analytics
  - Data is unbounded and infinite
  - Focuses on low latency
  - Supports event-driven architectures



Scalable Data Processing

Characteristics of Stream Processing

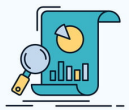
- Stream systems process data with minimal delay
- Designed for continuous operation
  - Records arrive continuously
  - State management is critical
  - Time semantics (event-time vs processing-time) matter



Scalable Data Processing

Use Cases of Stream Processing

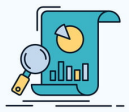
- Stream processing is used when timely insight is critical
  - Fraud detection in financial systems
  - Real-time monitoring and alerting
  - Live dashboards and recommendation systems



Scalable Data Processing

Challenges in Stream Processing

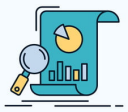
- Continuous processing introduces new challenges
  - Handling late or out-of-order data
  - Maintaining state across failures
  - Ensuring exactly-once or at-least-once guarantees



Scalable Data Processing

Batch vs Stream Processing – Conceptual View

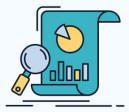
- Batch and Stream processing solve different problem types
  - Batch focuses on completeness and accuracy
  - Stream focuses on immediacy and responsiveness
  - Choice depends on business requirements



Scalable Data Processing

Batch vs Stream Processing – Comparison

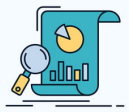
- Key differences between Batch and Stream processing
  - Data nature: Finite vs Infinite
  - Latency: High vs Low
  - Cost model: Scheduled vs Continuous



Scalable Data Processing

When to Choose Batch or Stream

- Choosing the correct paradigm is critical for system design
  - Use Batch when accuracy is more important than speed
  - Use Stream when real-time reaction is required
  - Many modern systems combine both

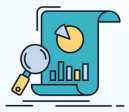


Scalable Data Processing

Introduction to Distributed File Systems

PES UNIVERSITY

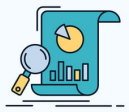
- Distributed File Systems (DFS) store data across multiple machines
- Designed to handle large datasets reliably
- Acts as the storage backbone for batch and stream systems
  - Data is split and distributed across nodes
  - Fault tolerance is built-in
  - Enables parallel data access



Scalable Data Processing

Why Distributed File Systems are Needed

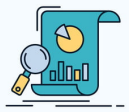
- Traditional file systems cannot scale for Big Data workloads
- DFS solves limitations of single-machine storage
  - Handles storage beyond single disk capacity
  - Provides high availability through replication
  - Supports distributed processing engines



Scalable Data Processing

Traditional FS vs Distributed FS

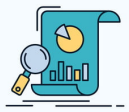
- Key differences between Traditional FS and DFS
  - Storage location: Single machine vs Cluster
  - Failure handling: Manual vs Automatic
  - Scalability: Vertical vs Horizontal



Scalable Data Processing

Core Characteristics of Distributed File Systems

- DFS are designed for reliability and scalability
  - Data replication for fault tolerance
  - High throughput access
  - Optimised for large files

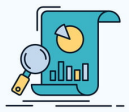


Scalable Data Processing

DFS and Data Processing Paradigms

PES UNIVERSITY

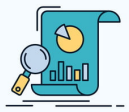
- DFS acts as the common storage layer
- Supports both batch and stream processing systems
  - Batch engines read large blocks sequentially
  - Stream systems may use DFS for checkpoints
  - Enables decoupling of storage and compute



Scalable Data Processing

Example: Unified Batch and Stream Pipeline

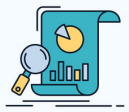
- Consider an online retail platform
- Same data is processed differently using batch and stream paradigms
  - Streaming pipeline generates real-time sales metrics
  - Batch pipeline computes daily and monthly summaries
  - Distributed File System stores raw and processed data



Scalable Data Processing

Importance of DFS in Scalable Systems

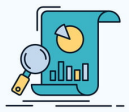
- DFS enables scalable analytics and processing
- Acts as a shared data layer across tools
  - Improves data locality
  - Enables parallel computation
  - Reduces system bottlenecks



Scalable Data Processing

Summary of Key Takeaways

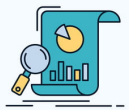
- Batch and Stream processing serve different needs
- Distributed File Systems are essential for scalability
- Modern data systems combine multiple paradigms



Scalable Data Processing

JOB Problem – Banana

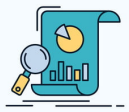
- Scenario: A university generates exam results once per semester
- Problem: Identify the suitable processing paradigm and justify
- Solution:
  - Batch processing is suitable
  - Data is finite and periodic
  - Low latency is not required



Scalable Data Processing

JOB Problem – Orange

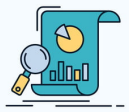
- Scenario: A ride-hailing app tracks live driver locations
- Problem: Choose processing paradigm and storage strategy
- Solution:
  - Stream processing for real-time updates
  - DFS used for historical trip storage



Scalable Data Processing

JOB Problem – Jackfruit

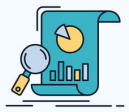
- Scenario: A banking system performs fraud detection and end-of-day reconciliation
- Problem: Design a combined processing architecture
- Solution:
  - Stream processing for fraud alerts
  - Batch processing for reconciliation
  - DFS as a shared storage layer



Scalable Data Processing
Brainstorming Questions

PES UNIVERSITY

- Can a system rely only on stream processing for all use cases?
- Why do batch systems still exist despite real-time technologies?
- How does DFS simplify distributed processing?
- What happens if storage is not fault-tolerant?



Scalable Data Processing

References

- Shilpi Saxena and Saurabh Gupta, *Practical Real-time Data Processing and Analytics*, Packt Publishing – Chapters 1 and 3
- Fabian Hueske and Vasiliki Kalavri, *Stream Processing with Apache Flink*, O'Reilly Media, 2019 – Chapter 1
- Apache Flink Documentation v1.20.0 – Concepts
- HDFS User Guide v3.4.0



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456



Scalable Data Processing

Distributed Data Processing

Niteesh K R

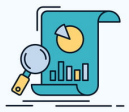
Computer Applications

Scalable Data Processing

Distributed Data Processing

Apache Hadoop and HDFS

Niteesh K R
Computer Applications

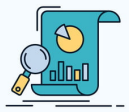


Scalable Data Processing

Introduction to Apache Hadoop Ecosystem

PES UNIVERSITY

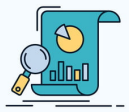
- Apache Hadoop is an open-source framework designed for storing and processing very large datasets in a distributed manner
- It allows data to be stored across clusters of machines and processed in parallel
- Hadoop was created to address scalability, fault tolerance, and cost limitations of traditional systems
  - Designed for TBs and PBs of data
  - Runs on clusters of commodity hardware
  - Software-based fault tolerance instead of specialised hardware



Scalable Data Processing

Why Hadoop Was Needed

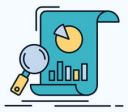
- As data volumes grew rapidly, traditional systems failed to scale efficiently
- Vertical scaling increased cost and complexity
- Hadoop introduced a distributed approach to both storage and computation
  - Single machines could not store or process all data
  - Hardware failures became frequent at large scale
  - Need for low-cost, fault-tolerant storage systems



Scalable Data Processing

Hadoop Design Philosophy

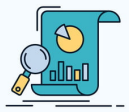
- Hadoop is built on the assumption that failures are normal, not exceptional
- Instead of avoiding failures, Hadoop detects and recovers from them
- Computation is moved closer to where data resides
  - Scale-out architecture instead of scale-up
  - Data locality improves performance
  - Replication ensures reliability



Scalable Data Processing

Hadoop Ecosystem Overview

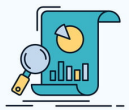
- Hadoop is an ecosystem composed of multiple components working together
- Each component addresses a specific aspect of distributed data handling
  - Storage of large datasets
  - Resource management and scheduling
  - Distributed batch processing
  - Supporting libraries and utilities



Scalable Data Processing

Core Components of Hadoop

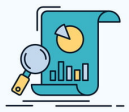
- Hadoop consists of four fundamental components
  - HDFS – Distributed file storage
  - YARN – Resource management and job scheduling
  - MapReduce – Distributed batch processing model
  - Hadoop Common – Shared utilities and libraries
- These components together form a complete distributed data platform



Scalable Data Processing

Role of HDFS in Hadoop

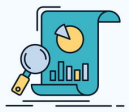
- HDFS acts as the primary storage layer in Hadoop
- All Hadoop processing frameworks read data from and write data to HDFS
  - Stores raw, intermediate, and processed data
  - Optimised for high-throughput access
  - Enables data locality for computation



Scalable Data Processing

Role of YARN (Conceptual)

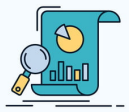
- YARN separates resource management from data processing
- It allows multiple applications to share the same cluster resources
  - Allocates CPU and memory dynamically
  - Manages application lifecycle
  - Enables scalability and multi-tenancy



Scalable Data Processing

Role of MapReduce (Conceptual)

- MapReduce is Hadoop's original processing framework
- It divides large jobs into smaller tasks executed in parallel
  - Map phase processes input splits
  - Reduce phase aggregates intermediate results
  - Suitable for large-scale batch workloads

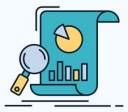


Scalable Data Processing

Introduction to HDFS

PES UNIVERSITY

- HDFS stands for Hadoop Distributed File System
- It is designed to store very large files across multiple machines
- Optimised for throughput rather than low-latency access
  - Files are split into large blocks
  - Blocks distributed across cluster nodes
  - Fault tolerance through replication

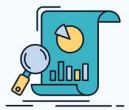


Scalable Data Processing

HDFS Design Goals

PES UNIVERSITY

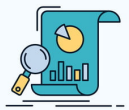
- HDFS is designed with clear architectural goals
  - Support very large datasets
  - Provide streaming access to data
  - Run efficiently on commodity hardware
- Certain trade-offs are accepted to achieve scalability



Scalable Data Processing

HDFS Block Concept

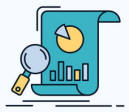
- Files in HDFS are divided into fixed-size blocks
- Block size is typically much larger than traditional file systems
  - Large blocks reduce metadata overhead
  - Enable parallel reads and writes
  - Improve overall throughput



Scalable Data Processing

HDFS Replication Mechanism

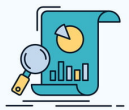
- Each HDFS block is stored as multiple replicas
- Replication provides fault tolerance and availability
  - Default replication factor is three
  - Replicas stored on different nodes
  - Automatic re-replication on failure



Scalable Data Processing

Write Once Read Many Model

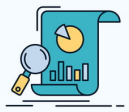
- HDFS follows a write-once, read-many access pattern
- Files are typically not modified after being written
  - Simplifies consistency management
  - Optimised for sequential reads
  - Ideal for analytical workloads



Scalable Data Processing

Traditional File System vs HDFS

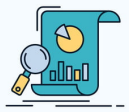
- HDFS differs fundamentally from traditional file systems
  - Traditional FS targets low latency
  - HDFS targets high throughput
  - Traditional FS assumes reliable hardware
  - HDFS assumes frequent failures



Scalable Data Processing

HDFS Use Cases

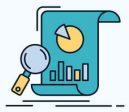
- HDFS is suitable for large-scale data storage and analytics
  - Log and clickstream storage
  - Data lakes and data warehouses
  - Machine learning training datasets



Scalable Data Processing

Limitations of HDFS

- HDFS is not designed for all workloads
  - Inefficient for small files
  - Poor support for random writes
  - Not suitable for low-latency applications

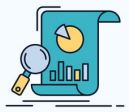


Scalable Data Processing

HDFS in Modern Architectures

PES UNIVERSITY

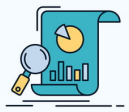
- HDFS often acts as a central data lake storage layer
- Multiple processing engines operate on the same data
  - Batch processing engines
  - Stream processing checkpoints
  - Shared analytics platforms



Scalable Data Processing

Summary of Key Concepts

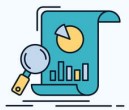
- Hadoop enables scalable distributed data storage and processing
- HDFS is the foundation of the Hadoop ecosystem
- Design trade-offs enable reliability and scalability at scale



Scalable Data Processing

JOB Problem – Banana

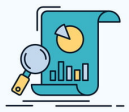
- Scenario: A company stores daily transaction logs for auditing
- Problem: Decide whether Hadoop and HDFS are suitable
- Solution:
  - HDFS is suitable for large log files
  - Batch processing supports auditing workloads



Scalable Data Processing

JOB Problem – Orange

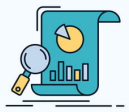
- Scenario: A startup expects rapid data growth over time
- Problem: Justify choosing HDFS over traditional file systems
- Solution:
  - Horizontal scalability
  - Built-in fault tolerance



Scalable Data Processing

JOB Problem – Jackfruit

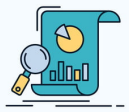
- Scenario: A data lake serves multiple analytics teams
- Problem: Design a Hadoop-based storage approach
- Solution:
  - Centralised HDFS storage
  - Shared access across engines
  - Data locality for performance



Scalable Data Processing

Brainstorming Questions

- Why does HDFS prioritise throughput over latency?
- What architectural changes are required to support small files efficiently?
- Can cloud object storage completely replace HDFS?



Scalable Data Processing

References

- Apache Hadoop Documentation v3.4.0
- HDFS User Guide v3.4.0
- Shilpi Saxena and Saurabh Gupta, *Practical Real-time Data Processing and Analytics*, Packt Publishing – Chapter 2



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456



Scalable Data Processing

Distributed Data Processing

Niteesh K R

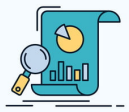
Computer Applications

Scalable Data Processing

Distributed Data Processing

HDFS – Architecture, Terms, Commands

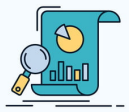
Niteesh K R
Computer Applications



Scalable Data Processing

HDFS Architecture – Overview

- HDFS follows a master–slave architecture to manage data at large scale
- The design separates metadata management from actual data storage
- This separation allows HDFS to scale horizontally while remaining manageable
  - Centralised metadata simplifies namespace management
  - Distributed storage enables parallel data access
  - Architecture assumes frequent hardware failures

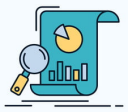


Scalable Data Processing

HDFS Architecture – Key Daemons

PES UNIVERSITY

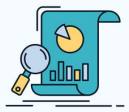
- HDFS architecture is implemented using specialised background services called daemons
- Each daemon has a clearly defined responsibility
  - NameNode – Maintains filesystem metadata and namespace
  - DataNode – Stores and manages actual data blocks
  - Secondary NameNode – Assists with metadata checkpointing
- This clear separation simplifies scalability and fault handling



Scalable Data Processing

NameNode – Central Role in HDFS

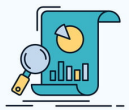
- NameNode is the master daemon and the most critical component of HDFS
- DOES NOT STORE ACTUAL DATA
- It manages all metadata related to the filesystem
- Client applications never interact directly with DataNodes without NameNode guidance
  - Maintains directory hierarchy and permissions
  - Maps files to blocks and blocks to DataNodes
  - Coordinates all read and write operations



Scalable Data Processing

Why NameNode Stores Metadata in Memory

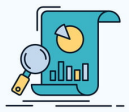
- NameNode stores metadata primarily in memory for performance reasons
- Fast metadata access is essential for high-throughput systems
- Disk-based metadata lookup would severely limit scalability
  - Enables millisecond-level file lookup
  - Improves overall cluster throughput
  - Introduces memory constraints for large namespaces
- This design explains why HDFS struggles with millions of small files



Scalable Data Processing

FsImage and EditLog

- HDFS metadata is persisted using two key files
- These files ensure durability of namespace information
  - FsImage – Snapshot of the complete filesystem metadata
  - EditLog – Sequence of recent changes since last snapshot
- During startup, NameNode reconstructs metadata using both files

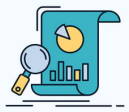


Scalable Data Processing

DataNode – Storage Backbone of HDFS

PES UNIVERSITY

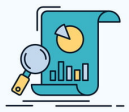
- DataNodes are responsible for storing actual file data
- Each DataNode manages data blocks on its local disk
- DataNodes are designed to be simple and replaceable
  - Serve read and write requests from clients
  - Perform block creation, deletion, and replication
  - Periodically report status to NameNode



Scalable Data Processing

Heartbeats and Block Reports

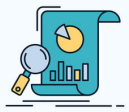
- DataNodes continuously communicate with the NameNode
- This communication is essential for fault detection
  - Heartbeats indicate that a DataNode is alive
  - Block reports list all blocks stored on the DataNode
  - Missing heartbeats trigger failure handling
- This mechanism allows HDFS to automatically recover from node failures



Scalable Data Processing

Secondary NameNode – The Misonomer

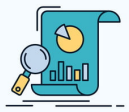
- Secondary NameNode is not a backup NameNode
- Its role is to manage metadata growth and stability
- It periodically performs checkpointing operations
  - Merges FsImage and EditLog
  - Produces a new compacted FsImage
  - Reduces NameNode restart time
- Upon NameNode Failure
  - All the metadata is in NameNode
  - Secondary NameNode has the timely backup of the NameNode metadata
  - Restarts the NameNode, by providing the metadata, doesn't replace the NameNode



Scalable Data Processing

HDFS Read Operation – Conceptual Flow

- HDFS read operation separates metadata lookup from data transfer
- Client first contacts NameNode for block location information
- Actual data transfer happens directly with DataNodes
  - Minimises NameNode load
  - Enables parallel data reads
  - Improves scalability

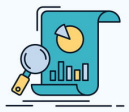


Scalable Data Processing

HDFS Write Operation – Conceptual Flow

PES UNIVERSITY

- Writing data to HDFS involves coordinated communication
- Data is written as a pipeline across multiple DataNodes
- Acknowledgements ensure reliable replication
  - NameNode selects DataNodes for replicas
  - Client streams data through pipeline
  - Failure triggers automatic recovery

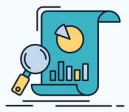


Scalable Data Processing

HDFS Block Size – Design Rationale

PES UNIVERSITY

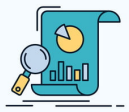
- HDFS uses very large block sizes compared to traditional file systems
- Large blocks reduce the amount of metadata stored in NameNode
- Optimised for sequential access patterns
  - Default block size is 128 MB
  - Fewer blocks reduce metadata overhead
  - Better throughput for large files



Scalable Data Processing

Trade-offs in Block Size Selection

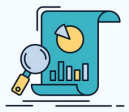
- Block size impacts performance and scalability
- There is no universally optimal block size
  - Larger blocks reduce NameNode memory usage
  - Smaller blocks increase parallelism
  - Improper sizing affects job efficiency



Scalable Data Processing

HDFS Replication – Why It Matters

- Replication is the primary mechanism for fault tolerance in HDFS
- Each block is stored on multiple DataNodes
- Failure of a node does not lead to data loss
  - Default replication factor is three
  - Ensures availability during failures
  - Enables parallel reads

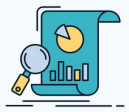


Scalable Data Processing

Replica Placement Strategy

PES UNIVERSITY

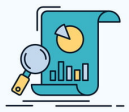
- HDFS uses rack-aware replica placement
- This strategy balances reliability and network efficiency
  - One replica on local node
  - One replica on a different rack
  - One replica on the same remote rack



Scalable Data Processing

HDFS Command Line Interface

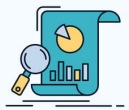
- HDFS provides a command-line interface similar to Unix
- Used for interacting with the HDFS namespace
  - File and directory management
  - Data ingestion and retrieval
  - Monitoring basic filesystem operations



Scalable Data Processing

Common HDFS Commands

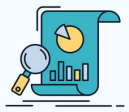
- List directory contents: `hdfs dfs -ls /`
- Create directories: `hdfs dfs -mkdir /input`
- Upload files: `hdfs dfs -put data.csv /input`
- Download files: `hdfs dfs -get /input/data.csv`



Scalable Data Processing

Data Loading into HDFS

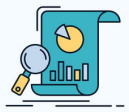
- Data loading is the entry point for Hadoop-based processing
- HDFS automatically handles block splitting and replication
  - Supports large-scale bulk ingestion
  - Handles unstructured and structured data
  - Transparent block management



Scalable Data Processing

Operational Challenges During Data Loading

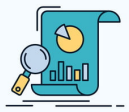
- Practical ingestion introduces operational challenges
  - Small file problem impacts NameNode memory
  - Network saturation during large uploads
  - Misconfigured replication affects reliability



Scalable Data Processing

Summary of HDFS Architecture and Operations

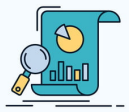
- HDFS separates metadata and data storage responsibilities
- Daemons coordinate to provide scalability and fault tolerance
- Block size and replication are critical tuning parameters



Scalable Data Processing

JOB Problem – Banana

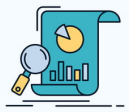
- Scenario: An organisation stores daily server logs in HDFS
- Problem: Suggest an appropriate block size strategy
- Solution:
  - Use default or larger block size
  - Logs are accessed sequentially



Scalable Data Processing

JOB Problem – Orange

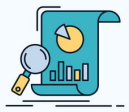
- Scenario: A DataNode crashes during an active write operation
- Problem: Explain how HDFS prevents data loss
- Solution:
  - Replication maintains multiple block copies
  - NameNode triggers re-replication



Scalable Data Processing

JOB Problem – Jackfruit

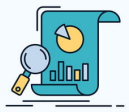
- Scenario: A dataset consists of millions of small files
- Problem: Analyse the impact on HDFS and propose solutions
- Solution:
  - Metadata overload on NameNode
  - Use HDFS archive or preprocessing techniques



Scalable Data Processing

Brainstorming Questions

- Why is NameNode considered a single point of concern?
- How does block size influence cluster performance?
- Is replication alone sufficient for data durability?



Scalable Data Processing

References

- Apache Hadoop Documentation v3.4.0
- HDFS User Guide v3.4.0
- Shilpi Saxena and Saurabh Gupta, *Practical Real-time Data Processing and Analytics*, Packt Publishing – Chapter 3



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456



Scalable Data Processing

Distributed Data Processing

Niteesh K R

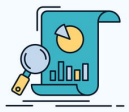
Computer Applications

Scalable Data Processing

Distributed Data Processing

EL I – Setting up HDFS, Commands

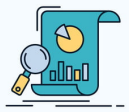
Niteesh K R
Computer Applications



Scalable Data Processing

Experiential Learning I – Overview

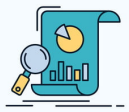
- Objective: To familiarise students with Hadoop setup and core HDFS commands
- Focus is on hands-on interaction with HDFS rather than theory
- Students will learn how to create directories, load data, inspect files, and manage HDFS storage
  - Understand Hadoop in practice
  - Gain confidence using HDFS CLI
  - Prepare for batch processing experiments



Scalable Data Processing

Hadoop Setup – Execution Mode

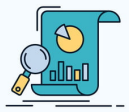
- Hadoop can run in different execution modes
- For laboratory sessions, pseudo-distributed mode is commonly used
  - Single machine acting as both NameNode and DataNode
  - HDFS runs as real daemons
  - Suitable for understanding HDFS behaviour
- Students should ensure Hadoop services are running before executing commands



Scalable Data Processing

Verifying Hadoop and HDFS Setup

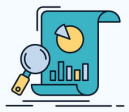
- Before interacting with HDFS, verify that Hadoop services are active
- Common checks include process verification and filesystem access
  - Use `jps` to list Hadoop daemons
  - Ensure NameNode and DataNode are running
  - Confirm HDFS is accessible using basic commands



Scalable Data Processing

HDFS Command Structure

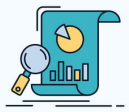
- HDFS commands follow a standard structure
- Commands are executed using the Hadoop filesystem shell
  - General syntax: `hdfs dfs <command> <arguments>`
  - Commands operate on HDFS namespace, not local filesystem
  - Behaviour is similar to Unix commands



Scalable Data Processing

Listing Files in HDFS – ls

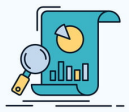
- The `-ls` command is used to list files and directories in HDFS
- It displays permissions, owner, size, and modification time
  - Command: `hdfs dfs -ls /`
  - Helps verify directory structure
  - First command students typically execute
- Internally, NameNode provides metadata for listing



Scalable Data Processing

Creating Directories – mkdir

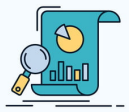
- Directories in HDFS are created using the `-mkdir` command
- Directory creation updates only metadata, not DataNodes
  - Command: `hdfs dfs -mkdir /data`
  - Parent directories must exist unless `-p` is used
  - No physical storage until files are added



Scalable Data Processing

Uploading Data to HDFS – put

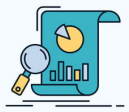
- The `-put` command uploads files from local filesystem to HDFS
- This is the primary method for loading data into Hadoop
  - Command: `hdfs dfs -put local.txt /data`
  - File is split into blocks automatically
  - Blocks are replicated across DataNodes
- NameNode coordinates block placement during upload



Scalable Data Processing

Alternative Upload Command – copyFromLocal

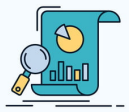
- `-copyFromLocal` is functionally similar to `-put`
- Explicitly indicates source is local filesystem
  - Command: `hdfs dfs -copyFromLocal file.csv /input`
  - Commonly used in scripted workflows
  - Improves readability in automation



Scalable Data Processing

Viewing File Contents – cat

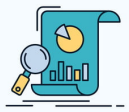
- The `-cat` command displays file contents on the terminal
- Useful for verifying data upload
  - Command: `hdfs dfs -cat /data/file.txt`
  - Reads data directly from DataNodes
  - Not suitable for large files



Scalable Data Processing

Viewing File Contents – head and tail

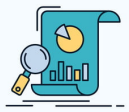
- HDFS supports partial file viewing using `-head` and `-tail`
- Useful for inspecting large files
  - `hdfs dfs -head /data/file.txt`
  - `hdfs dfs -tail /data/file.txt`
  - Avoids loading entire file



Scalable Data Processing

File and Directory Information – du and df

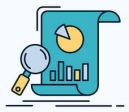
- `-du` shows space used by files and directories
- `-df` shows available space in HDFS
  - `hdfs dfs -du /data`
  - `hdfs dfs -df -h`
  - Helps monitor storage usage



Scalable Data Processing

Downloading Data from HDFS – get

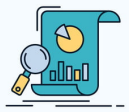
- The `-get` command copies files from HDFS to local filesystem
- Used for result retrieval and debugging
  - Command: `hdfs dfs -get /data/file.txt`
  - Reads blocks from nearest DataNode
  - Preserves original data in HDFS



Scalable Data Processing

Copying Files within HDFS – cp

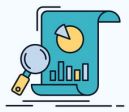
- `-cp` is used to copy files within HDFS
- No local filesystem involvement
  - Command: `hdfs dfs -cp /data/a.txt /backup`
  - Metadata updated at NameNode
  - Data blocks reused where possible



Scalable Data Processing

Moving and Renaming Files – mv

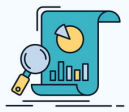
- `-mv` moves or renames files within HDFS
- Operation affects metadata only
  - Command: `hdfs dfs -mv /data/a.txt /data/b.txt`
  - Fast operation
  - No data copying involved



Scalable Data Processing

Uploading Data – copyFromLocal

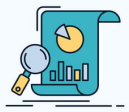
- The `-copyFromLocal` command uploads files from the local filesystem to HDFS
- It is functionally similar to `-put`, but explicitly indicates the source location
  - Command: `hdfs dfs -copyFromLocal sales.csv /input`
  - Preferred in scripted or automated workflows
  - Improves clarity when mixing local and HDFS paths
- Internally, the file is split into blocks and replicated across DataNodes



Scalable Data Processing

Uploading and Removing Source – moveFromLocal

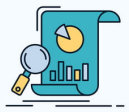
- The `-moveFromLocal` command uploads a file and deletes it from the local filesystem
- Useful when local storage must be freed after ingestion
  - Command: `hdfs dfs -moveFromLocal logs.txt /archive`
  - Combines upload and cleanup in a single step
  - Prevents duplicate storage of the same data
- Once moved, the file exists only inside HDFS



Scalable Data Processing

Downloading Data – copyToLocal

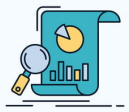
- The `-copyToLocal` command copies files from HDFS to the local filesystem
- The original file remains unchanged in HDFS
  - Command: `hdfs dfs -copyToLocal /output/result.txt`
  - Used for result inspection and debugging
  - Safe operation that does not affect HDFS data
- Data is read from the nearest available DataNode



Scalable Data Processing

Downloading and Removing Source – moveToLocal

- The `-moveToLocal` command moves files from HDFS to the local filesystem
- The file is deleted from HDFS after successful transfer
  - Command: `hdfs dfs -moveToLocal /temp/data.txt`
  - Useful for archival or one-time data extraction
  - Frees HDFS storage space
- Operation involves metadata update and block deletion in HDFS

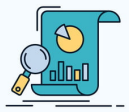


Scalable Data Processing

Choosing the Right Data Transfer Command

PES UNIVERSITY

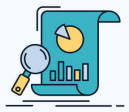
- HDFS provides multiple commands to support different data movement needs
  - Use `-copyFromLocal` when local data must be retained
  - Use `-moveFromLocal` when local data should be removed
  - Use `-copyToLocal` for safe result inspection
  - Use `-moveToLocal` to permanently extract data
- Correct command selection avoids accidental data loss



Scalable Data Processing

Deleting Files and Directories – rm

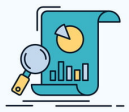
- `-rm` removes files and directories from HDFS
- Deletion updates metadata and frees blocks
  - `hdfs dfs -rm /data/file.txt`
  - `hdfs dfs -rm -r /data`
  - Deleted blocks are reclaimed by DataNodes



Scalable Data Processing

Changing Permissions – chmod

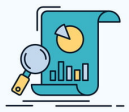
- HDFS supports Unix-like permission model
- `-chmod` modifies access permissions
  - Command: `hdfs dfs -chmod 755 /data`
  - Controls read, write, execute access
  - Enforced by NameNode



Scalable Data Processing

Checking File Replication – stat

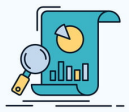
- File metadata can be inspected using `-stat`
- Useful for verifying replication and block size
  - Command: `hdfs dfs -stat %r /data/file.txt`
  - Displays replication factor
  - Helps validate HDFS configuration



Scalable Data Processing

Experiential Learning – Outcome

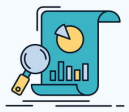
- Students gain hands-on experience with HDFS
- Core commands build confidence in Hadoop usage
- Prepares students for MapReduce and Spark labs



Scalable Data Processing

JOB Problem – Banana

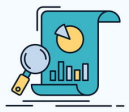
- Task: Create a directory and upload a sample dataset
- Expected Outcome: Correct directory structure and file presence



Scalable Data Processing

JOB Problem – Orange

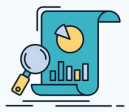
- Task: Inspect storage usage and replication of uploaded files
- Expected Outcome: Correct interpretation of `du` and `stat`



Scalable Data Processing

JOB Problem – Jackfruit

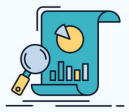
- Task: Simulate incorrect deletion and recovery strategy
- Expected Outcome: Understanding metadata and block lifecycle



Scalable Data Processing

Brainstorming Questions

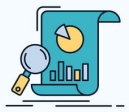
- Why are HDFS commands slower than local filesystem commands?
- Which operations modify metadata only and why?
- How does HDFS ensure consistency during concurrent access?



Scalable Data Processing

HDFS Failure and Recovery – Objective

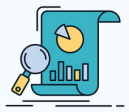
- Objective: To understand how HDFS handles node failures transparently
- This demonstration focuses on DataNode failures and recovery mechanisms
- Students will observe how replication ensures fault tolerance
  - Simulate failures without data loss
  - Understand NameNode decision-making
  - Observe automatic recovery



Scalable Data Processing

Why Failure Handling Is Critical in HDFS

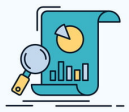
- HDFS is designed to run on commodity hardware
- Hardware failures are expected, not exceptional
  - Disk crashes
  - Network partitions
  - Power failures
- HDFS prioritises availability and durability over strict consistency



Scalable Data Processing

Types of Failures in HDFS

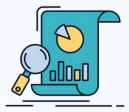
- HDFS primarily deals with two failure categories
  - DataNode failure – loss of block replicas
  - Network failure – temporary loss of connectivity
- NameNode failures are handled separately using standby mechanisms



Scalable Data Processing

Heartbeat and Block Reports

- DataNodes periodically send heartbeat messages to the NameNode
- Block reports contain information about stored block replicas
  - Heartbeat interval is typically a few seconds
  - Missing heartbeats indicate possible failure
  - Block reports help maintain metadata accuracy
- NameNode relies on these signals to detect failures

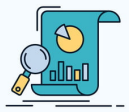


Scalable Data Processing

Simulating a DataNode Failure

PES UNIVERSITY

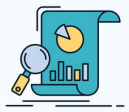
- A DataNode failure can be simulated by stopping the daemon
- This does not immediately remove metadata
  - Command example: `stop-dfs.sh` (partial or node-specific)
  - NameNode waits for heartbeat timeout
  - Blocks are marked under-replicated



Scalable Data Processing

Detection of DataNode Failure

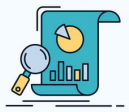
- If heartbeats stop beyond a threshold, the DataNode is marked dead
- The NameNode updates its internal metadata
  - Blocks on failed node are considered unavailable
  - Replication factor may fall below desired value
  - Recovery process is triggered automatically



Scalable Data Processing

Re-Replication of Blocks

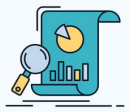
- NameNode schedules re-replication for under-replicated blocks
- New replicas are created on healthy DataNodes
  - Source replica chosen based on proximity
  - Target DataNodes selected using rack awareness
  - Replication happens in the background
- Applications continue accessing data during recovery



Scalable Data Processing

Rack Awareness in Recovery

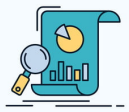
- HDFS uses rack awareness to avoid correlated failures
- Replicas are distributed across racks
  - Protects against switch or rack failures
  - Improves network efficiency
  - Ensures higher availability



Scalable Data Processing

Observing Recovery in Practice

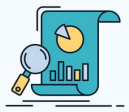
- Recovery can be observed using filesystem statistics
- Useful commands during demonstration
  - `hdfs dfsadmin -report`
  - `hdfs fsck /`
  - `hdfs dfs -stat %r <file>`
- These commands reflect real-time cluster state



Scalable Data Processing

Safe Mode During Recovery

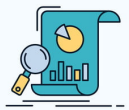
- HDFS may enter Safe Mode during startup or severe failures
- In Safe Mode, replication and deletion are suspended
  - Allows metadata stabilisation
  - Prevents accidental data loss
  - Exits automatically when conditions are met



Scalable Data Processing

Limitations of HDFS Failure Recovery

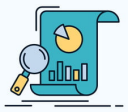
- HDFS recovery is effective but not instantaneous
- Some limitations remain
  - Small files stress NameNode memory
  - Re-replication consumes network bandwidth
  - Not suitable for low-latency updates



Scalable Data Processing

Experiential Learning – Takeaways

- HDFS assumes failures and handles them automatically
- Replication is central to fault tolerance
- Recovery is transparent to applications
- NameNode plays a critical coordination role



Scalable Data Processing

References

- Apache Hadoop Documentation v3.4.0
- HDFS User Guide v3.4.0



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456



Scalable Data Processing

Distributed Data Processing

Niteesh K R

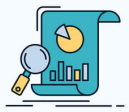
Computer Applications

Scalable Data Processing

Distributed Data Processing

MapReduce – Introduction, Phases, Implementation

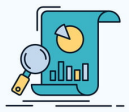
Niteesh K R
Computer Applications



Scalable Data Processing

MapReduce – Introduction

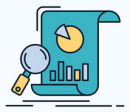
- MapReduce is a distributed programming model designed for large-scale data processing
- It allows computation to be expressed as two primary operations: Map and Reduce
- The model abstracts away complexities of parallelisation, fault tolerance, and data distribution
  - Introduced by Google for processing web-scale data
  - Adopted by Hadoop as its core batch processing paradigm



Scalable Data Processing

Why MapReduce Was Needed

- Traditional single-machine processing does not scale for massive datasets
- Distributed processing introduces challenges
  - Task coordination
  - Failure handling
  - Data movement and locality
- MapReduce addresses these challenges through a simple and structured execution model

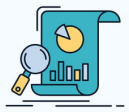


Scalable Data Processing

MapReduce Programming Model

PES UNIVERSITY

- MapReduce programs are expressed using key–value pairs
- Both input and output of each phase follow the same abstraction
  - Mapper: `(key, value)` → `(key, value)`
  - Reducer: `(key, list(values))` → `output`
- This uniform structure enables scalable parallel execution

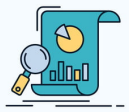


Scalable Data Processing

Core Components of MapReduce

PES UNIVERSITY

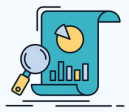
- MapReduce execution involves multiple logical components
  - Input Splits
  - Mapper Tasks
  - Shuffle and Sort
  - Reducer Tasks
- Hadoop manages task scheduling and execution automatically



Scalable Data Processing

Phase 1 – Input Splitting

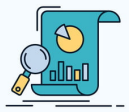
- Input data stored in HDFS is divided into fixed-size splits
- Each split is processed independently by a mapper
  - Typically aligned with HDFS block size
  - Enables parallelism
  - Improves data locality



Scalable Data Processing

Phase 2 – Mapping

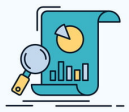
- The mapper processes input records independently
- It transforms raw data into intermediate key–value pairs
  - Stateless processing
  - Highly parallel execution
  - No dependency on other mappers
- Mapping focuses on data transformation, not aggregation



Scalable Data Processing

Phase 3 – Shuffle and Sort

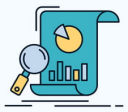
- Intermediate outputs from mappers are grouped by key
- Hadoop automatically performs shuffle and sort
  - Transfers data across nodes
  - Groups values for each unique key
  - Sorts keys before reduction
- This phase is critical for correct aggregation



Scalable Data Processing

Phase 4 – Reducing

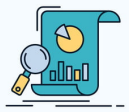
- Reducers process grouped key–value pairs
- Aggregation or summarisation is performed here
  - One reducer per key group
  - Can produce final or intermediate output
- Reducer output is written back to HDFS



Scalable Data Processing

MapReduce Execution Flow

- Entire workflow is managed by Hadoop
  - Job submission
  - Task scheduling via YARN
  - Monitoring and retries on failure
- Developers focus only on Map and Reduce logic

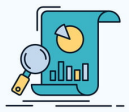


Scalable Data Processing

Why Use Python for MapReduce?

PES UNIVERSITY

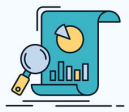
- Hadoop Streaming allows MapReduce programs in non-Java languages
- Python is ideal for teaching and rapid development
  - Minimal syntax overhead
  - Clear focus on logic
  - Easy debugging



Scalable Data Processing

Hadoop Streaming – Concept

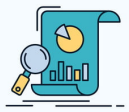
- Hadoop Streaming treats mapper and reducer as executable scripts
- Data is passed using standard input and output
  - Mapper reads from `stdin`
  - Reducer receives grouped input via `stdin`
- Hadoop handles splitting, grouping, and distribution



Scalable Data Processing

Python MapReduce Example – Problem Statement

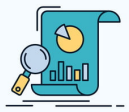
- Problem: Count the frequency of each word in a large text dataset
- Dataset is stored in HDFS
- Objective is to demonstrate complete MapReduce execution
  - Map: Emit (word, 1)
  - Reduce: Sum counts per word



Scalable Data Processing

Python Mapper Logic (Conceptual)

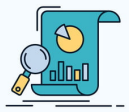
- Mapper reads input line by line
- Each line is tokenised into words
- Emits intermediate key–value pairs
  - Output format: `word\t1`
  - One output per word occurrence



Scalable Data Processing

Python Reducer Logic (Conceptual)

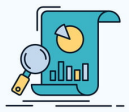
- Reducer receives sorted key–value pairs
- Aggregates counts for each word
  - Maintains running total
  - Outputs final count per word



Scalable Data Processing

End-to-End Execution

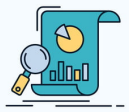
- Steps involved in execution
  - Upload dataset to HDFS
  - Submit Hadoop Streaming job
  - Inspect output in HDFS
- Entire pipeline executes in a distributed manner



Scalable Data Processing

Strengths of MapReduce

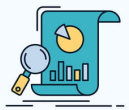
- Simple and scalable processing model
- Built-in fault tolerance
- Suitable for batch-oriented workloads



Scalable Data Processing

Limitations of MapReduce

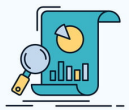
- High latency due to disk I/O between phases
- Not suitable for iterative or real-time processing
- Complex workflows require multiple jobs



Scalable Data Processing

JOB Problem – Banana

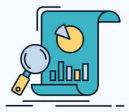
- Write a mapper that emits (length, 1) for each word
- Reduce to count words by length



Scalable Data Processing

JOB Problem – Orange

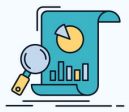
- Modify reducer to emit only words with frequency greater than 10



Scalable Data Processing

JOB Problem – Jackfruit

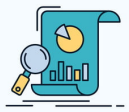
- Design a two-stage MapReduce workflow for log analysis
- Justify job chaining and intermediate outputs



Scalable Data Processing

Brainstorming Questions

- Why is shuffle and sort the most expensive phase?
- How does MapReduce achieve fault tolerance?
- Why did Spark replace MapReduce for many workloads?



Scalable Data Processing

References

- Apache Hadoop Documentation v3.4.0
- Hadoop MapReduce Tutorial v3.4.0



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456



Scalable Data Processing

Distributed Data Processing

Niteesh K R

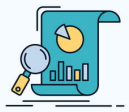
Computer Applications

Scalable Data Processing

Distributed Data Processing

JOB Problems – I

Niteesh K R
Computer Applications

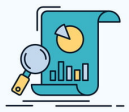


Scalable Data Processing

JOB Problems – Unit I (Overview)

PES UNIVERSITY

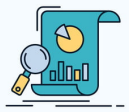
- JOB Problems are designed to assess conceptual clarity and applied understanding
- These problems move beyond definitions and require reasoning
- Difficulty levels are categorised as Banana, Orange, and Jackfruit
  - Banana – Basic understanding
  - Orange – Applied reasoning
  - Jackfruit – System-level thinking



Scalable Data Processing

JOB Problem – Banana 1

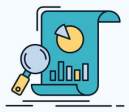
- Scenario: A university stores student attendance records for the last 10 years
- Problem: Identify whether this is a batch or stream processing use case
- Justify your answer
- Expected Thinking:
  - Data is historical and static
  - Processing is periodic, not continuous



Scalable Data Processing

JOB Problem – Banana 2

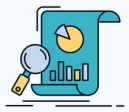
- Scenario: A company wants to store large CSV files (100 GB each) reliably
- Problem: Why is a Distributed File System preferred over a traditional file system?
- Expected Thinking:
  - Fault tolerance through replication
  - Parallel data access
  - Scalability using commodity hardware



Scalable Data Processing

JOB Problem – Orange 1

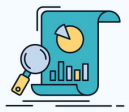
- Scenario: An e-commerce platform analyses daily sales at midnight
- Problem:
  - Identify the processing model used
  - Explain why stream processing is not required here
- Expected Thinking:
  - Batch processing suits periodic aggregation
  - Low latency is not a requirement



Scalable Data Processing

JOB Problem – Orange 2

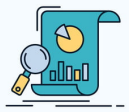
- Scenario: A file stored in HDFS has a replication factor of 3
- One DataNode fails unexpectedly
- Problem:
  - Explain how HDFS detects the failure
  - Describe how data availability is maintained
- Expected Thinking:
  - Heartbeats and block reports
  - Automatic re-replication



Scalable Data Processing

JOB Problem – Jackfruit 1

- Scenario: A social media company processes both historical posts and live user activity
- Problem:
  - Propose an overall data processing architecture
  - Clearly identify batch and stream components
- Expected Thinking:
  - Batch processing for historical analytics
  - Stream processing for real-time events
  - Unified storage using distributed systems



Scalable Data Processing

JOB Problem – Jackfruit 2

- Scenario: A dataset contains millions of small files stored in HDFS
- Problem:
  - Analyse the impact on NameNode performance
  - Suggest architectural or design-level solutions
- Expected Thinking:
  - Metadata overhead in memory
  - Use of file consolidation or alternate storage models



Thank You

Niteesh K R

Assistant Professor

Department of Computer Applications

niteeshkr@pes.edu

080-26721983 Extn: 456